

OpenClaw harness '1, içeriden

Bir harness'ın gerçekten ne yaptığı: bozuk tool çağrısını onarmak, gürültülü çıktıyı kesmek, koşan turu yönlendirmek, düzeltmeyi skill'e çevirmek. Her slaytta mekanizma ve nasıl uygulandığı.

[@01cc7106](https://github.com/openclaw/openclaw) · 22 paket · 161 extension

kavramsal anlatım: [hap-openclaw.pdf](#) · ölçümler: [docs/pdf/openclaw-ici.pdf](#)

Paket haritası

ai · llm-core · model-catalog-core	sağlayıcı adaptörleri, akış runtime'ı	118
gateway-protocol · gateway-client · sdk	tipli şema + doğrulayıcı + istemci	108
memory-host-sdk	bellek sağlayıcı sözleşmesi	83
plugin-sdk · plugin-package-contract	eklenti yüzeyi	25
terminal-core · markdown-core · media-*	çıkış biçimlendirme	21
tool-call-repair · retry · net-policy	sessiz altyapı	6

22 paket · dosya sayıları en büyük modülü gösteriyor. Çekirdeğin her ilginç parçası ayrı paket — sıkıştırma bile.

22 paket · çekirdeğin her ilginç parçası ayrı

Bir sistemin nasıl düşünüldüğünü öğrenmenin en hızlı yolu, kodun nasıl bölündüğüne bakmaktır. OpenClaw 22 pakete bölünmüş. En büyük üçü şunlar: **ai** (118 dosya) model sağlayıcılarıyla konuşuyor, **gateway-protocol** (108 dosya) kontrol düzleminin tipli şemasını tutuyor, **memory-host-sdk** (83 dosya) bellek sağlayıcılarının uyması gereken sözleşmeyi tanımlıyor. Bu üçünün ayrı olması bir tercih: model erişimi, kontrol düzlemi ve bellek **birbirinden bağımsız değişebiliyor**. Sağlayıcı eklemek kontrol düzlemine dokunmuyor. İlginç olan ise en küçükler. `tool-call-repair` 6 dosya, `retry` tek dosya. Ama ikisi de olmadığında sistem çökmüyor, **sessizce yanlış çalışıyor** — ki bu daha kötü.

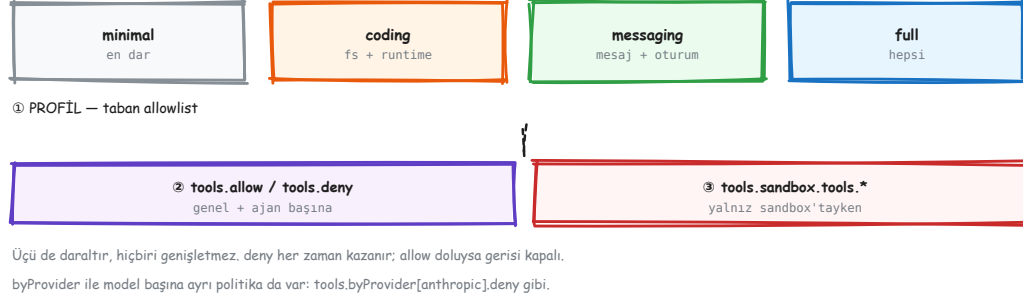
Built-in tool kataloğu

group:fs	4	read write edit apply_patch
group:runtime	3	exec process code execution
group:web	3	web_search web_fetch x_search
group:memory	2	memory_search memory_get
group:sessions	15	sessions{,_list,_history,_search,_send,_spawn,_yield} conversations {list,send,turn} agents_wait subagents session_status {spawn,dis
group:ui	6	browser screen dashboard terminal canvas show_widget
group:messaging	1	message
group:automation	2	heartbeat_respond gateway
group:nodes	3	nodes computer mobile_ui
group:agents	7	agents_list get_goal create_goal update_goal update_plan ask_user skill_workshop
group:media	5	image image_generate music_generate video_generate tts

51 tool · 11 somut grup · src/agents/tool-catalog.ts

Bir ajanın ne yapabildiğini, elindeki tool listesi belirler. OpenClaw'ın kutudan çıkan listesi **51 tool**, on bir grupta toplanmış. Asıl bilgi sayıda değil **dağılımda**. En kalabalık grup 15 tool'la **sessions**: alt-ajan başlatmak, ona iş devretmek, cevabını beklemek, aralarında mesajlaşmak. Buna karşılık dosya işlemleri için 4, komut çalıştırmak için 3 tool var. Yani yatırımın büyük kısmı dosyaya ya da kabuğa değil, **ajanların birbirini yönetmesine** yapılmış. Bu da tasarımın niyetini söylüyor: tek bir asistan değil, **çok ajanlı bir işletim ortamı** kurmuşlar.

Üç sayı, üç anlam



51 kaynakta · 44 canlı kurulumda · doküman tablosu eskimiş

"Kaç tool var?" sorusuna üç farklı cevap geliyor ve üçü de doğru, çünkü üçü farklı şeyi sayıyor. **51**, kaynak kodda tanımlı tool sayısı. **44**, bizim ölçtüğümüz canlı gateway'in gerçekten sunduğu sayı; aradaki fark filtrelerde eriyor. Dokümandaki tablo ise üçüncü bir sayı veriyor ve o **eskimiş**: listede olmayan `agents_wait`, `dashboard`, `mobile_ui` eksik, buna karşılık artık var olmayan bir `cron` tool'u duruyor. Daralma üç aşamada oluyor. Profil bir taban liste veriyor, `allow/deny` onu kesiyor, sandbox'tayken sandbox politikası bir kez daha kesiyor. Üçü de yalnız **daraltıyor**; hiçbiri listeye tool ekleyemiyor.

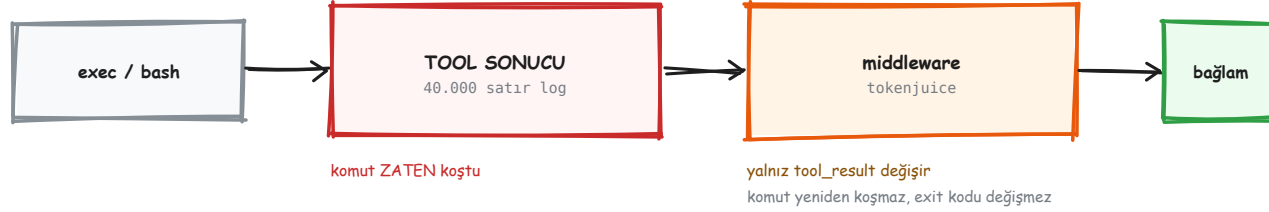
Tool Call Repair – bozuk çağrıyı kurtarmak



packages/tool-call-repair – ayrıştır → ada eşle → gerçek çağrıya çevir

Modelin bir tool çağırmasının doğru yolu, sağlayıcının function calling biçimini kullanmaktır. Ama bazı modeller bunu beceremiyor: tool çağrısını cevabın içine **düz yazı olarak** yazıyorlar. OpenClaw bu düz yazıyı tanıyıp gerçek çağrıya çeviriyor. Dört farklı biçim tanınıyor: `[END_TOOL_REQUEST]` bloğu, Harmony'nin `<|channel|>` / `<|message|>` / `<|call|>` işaretçileri, ve XML'e benzeyen `<function=ad>` etiketleri. Asıl kritik adım ikincisi: modelin yazdığı ad, sağlayıcının *o istekte* izin verdiği tam tool adına eşleniyor. Yakın ama birebir olmayan bir ad işe yaramaz. Onarmazsan tur boşa gider ve **hiçbir yerde sebebi görünmez**.

Tool sonucu ara katmanı

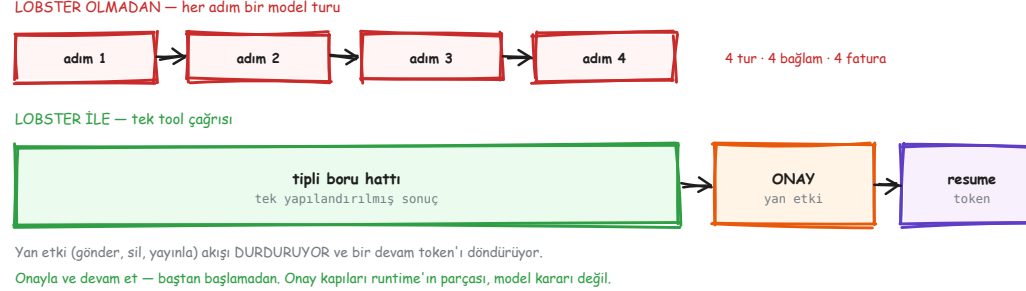


Ayrım önemli: bu bir çıktı KISALTMA katmanı, bir komut değiştirme katmanı değil. Gürültülü log bağlamı yemeden önce kesiliyor.

tokenjuice – komuta değil, SONUCA dokunuyor

Bir `exec` çağrısı 40.000 satır log döndürebilir. O log olduğu gibi bağlama girerse tek bir komut bütün pencereyi yer. Tokenjuice bu sorunu, komut **zaten koştuktan sonra** çözüyor: çıktıyı kısaltıyor. Hangi katmanda durduğu tasarımın kendisi. Kabuğa giden komutu yeniden yazmıyor, komutu tekrar koşturmuyor, çıkış kodunu değiştirmiyor. Yalnızca modele dönen `tool_result` 'a dokunuyor. Sebebi şu: komutu değiştirseydin sonucun *doğruluğuna* karışmış olurdun. Sonucu kısaltmak ise yalnızca **bağlam muhasebesi** — ne koştuğu değişmiyor, modelin ne kadarını gördüğü değişiyor.

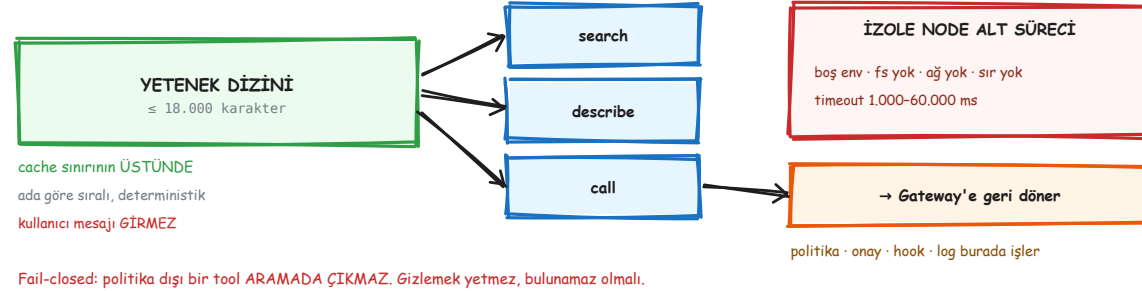
Lobster – tipli iş akışı runtime'ı



tek tool çağrısı · gömülü onay kapıları · devam token'ı

Çok adımlı bir işi modele orkestra ettirirsen her adım ayrı bir tur olur. Dört adımlı bir iş, dört model turu demektir; her turda bütün bağlam yeniden gönderilir. Lobster bu orkestrasyonu modelden alıp **tipli bir runtime'a** veriyor. Model tek bir çağrı yapıyor, runtime bütün boru hattını koşturuyor, geriye tek bir yapılandırılmış sonuç dönüyor. İçinde onay kapıları da var. Bir adım yan etkiliyse (mesaj gönder, yayınla, sil) akış **orada duruyor** ve bir devam token'ı döndürüyor. Onayladıktan sonra baştan başlamıyorsun, kaldığı yerden devam ediyor. Önemli olan şu: **kapıyı runtime tutuyor, model değil.** Model "onay sormayı atlayayım" diye karar veremiyor.

Code Mode ve Swarm



openclaw.tools.* köprüsü – izole alt süreç, sır yok

Katalog büyüdükçe bütün tool şemalarını prompt'a koymak imkânsızlaşıyor. **Code Mode** bunu şöyle çözüyor: model tool şemalarını görmüyor, bunun yerine küçük bir JavaScript köprüsüne `search`, `describe`, `call` yazıyor. **Swarm** aynı köprüden eşzamanlı alt-ajanlar başlatıyor ve sonuçlarını yapılandırılmış biçimde topluyor. İkisinin güvenlik hikâyesi aynı ve dikkat çekici. Köprü, izole bir Node alt sürecinde koşuyor: **environment boş, dosya sistemi yok, ağ yok, eklenti kodu yok, sır yok.** Yani köprüde koşan kod tek başına hiçbir şey yapamıyor. Gerçek her çağrı Gateway'e geri dönüyor ve normal politika, onay, hook ve log yolundan geçiyor.

Koşan bir tura müdahale etmenin dört yolu

/steer	koşan turu yönlendir	kabul edilmezse normal prompt olarak gider
/btw	yan soru	geçmişe EKLENMEZ · aynı model · tek atış
/goal	oturuma bağlı hedef	kalıcı · operatör ve model aynı hedefi görür
/loop	kendi kendini tekrarlayan iş	sahibe özel · konuşmaya bağlı

Dördü de aynı fikrin farklı yüzü: BİR TURA MÜDAHALE ETMEK. Klasik sohbet arayüzünde bunların hiçbiri yok.

/steer · /btw · /goal · /loop

Sıradan bir sohbet arayüzünde mesajı gönderdikten sonra yapabileceğin tek şey beklemektir. Bu dört komut aynı soruya farklı cevaplar veriyor: **tur çoktan başlamışken ne yapabilirsin?** /steer koşan turu yönlendiriyor. Runtime müdahaleyi kabul etmezse mesajı çöpe atmıyor, sıradan bir prompt olarak gönderiyor. /btw araya bir yan soru sokuyor ve cevabı **konuşma geçmişine eklemiyor**, yani asıl işin bağlamını kirletmiyor. /goal oturuma kalıcı bir hedef bağlıyor; hem operatör hem model aynı hedefi görüyor. /loop ise konuşmaya bağlı, kendini tekrarlayan bir iş kuruyor.

Self-learning – düzeltmeyi skill'e çevirmek



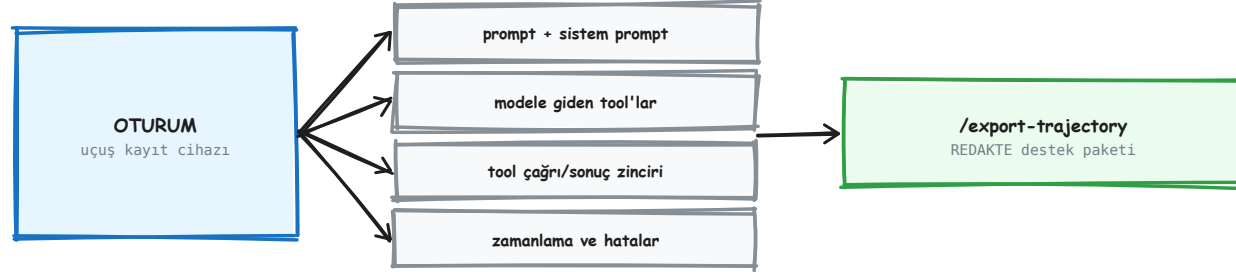
Kalıcı birim SKILL — bir bellek satırı değil. Fark: skill bir PROSEDÜR, gelecekteki oturumlar bulup izleyebiliyor.

Ve kapıdan geçiyor: öğrenilen şey doğrudan davranışa yazılmıyor, önerilip taranıp uygulanıyor.

Skill Workshop: öner → tara → uygula → yaşam döngüsü

Bir asistana "öyle değil, böyle yapacaksın" dediğinde o bilgi nereye gidiyor? Akla ilk gelen cevap belleğe yazmak. Ama bellekteki bir satır bir *olgudur*; sonraki oturumun izleyeceği bir *prosedür* değildir. "Kullanıcı tabloları sever" ile "rapor yazarken önce şunu, sonra şunu yap" aynı şey değil. OpenClaw'ın cevabı şu: kalıcı birim **skill**. Bir düzeltme ya da başarıyla biten bir iş, Skill Workshop'un yönetilen yolundan geçip yeniden kullanılabilir bir prosedüre dönüşüyor. Elle yazılan skill'ler de **aynı** yoldan geçiyor. Kritik olan aradaki kapı: öğrenilen şey doğrudan davranışa yazılmıyor. Önce öneriliyor, sonra taranıyor, sonra uygulanıyor.

Trajectory – oturumun uçuş kayıt cihazı

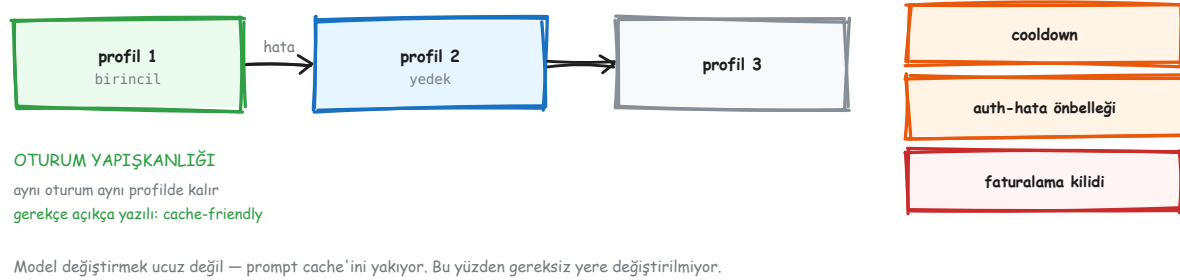


Bir hata raporunun "ne oldu" sorusunu prompt'u elle kopyalamadan cevaplıyor — ve redakte edilmiş çıkıyor.

/export-trajectory → redakte edilmiş destek paketi

"Ajan neden bunu yaptı?" sorusunun cevabı normalde elde yoktur. Modele tam olarak ne gittiğini görmek için prompt'u elle yeniden kurman gerekir. Trajectory bunu baştan kaydediyor. Her ajan koşusu için yapılandırılmış bir zaman çizelgesi tutuluyor: modele giden prompt ve sistem prompt'u, gönderilen tool tanımları, tool çağrı ve sonuç zinciri, süreler ve hatalar. **/export-trajectory** bu kaydı tek komutla bir destek paketine çeviriyor. Ve **redaksiyon varsayılan**. Bu küçük bir ayrıntı gibi duruyor ama değil: hata raporu paylaşmanın sır paylaşmak anlamına gelmemesi, insanların gerçekten rapor göndermesini sağlayan şey.

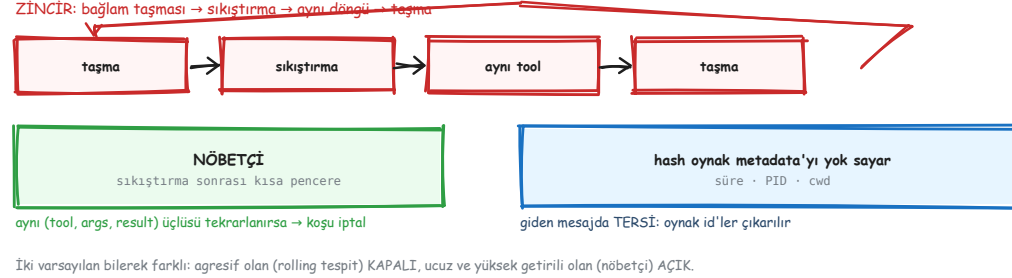
Model failover



profil rotasyonu · cooldown · auth-hata önbellegi · oturum yapışkanlığı

Bir model sağlayıcısı düştüğünde ne oluyor? Sırayla başka profiller deneniyor. Ama körlemesine değil, üç frenle. **Cooldown**, az önce düşen profili bir süre atlıyor. **Auth-hata önbellegi**, aynı 401'i tekrar tekrar yemeyi engelliyor. **Faturalama kilidi**, kotası bitmiş profili devre dışı bırakıyor. En ilginç ayrıntı sonuncusu: **oturum yapışkanlığı**. Aynı oturum, mümkün olduğunca aynı profilde kalmaya çalışıyor ve gerekçesi belgede açıkça yazılı — *cache-friendly*. Model değiştirmek prompt cache'ini yakıyor, çünkü cache isteğin başındaki değişmeyen kısımdan çalışıyor ve model değişince o kısım da değişiyor. Yani gereksiz yere model değiştirmemek bir **maliyet** kararı.

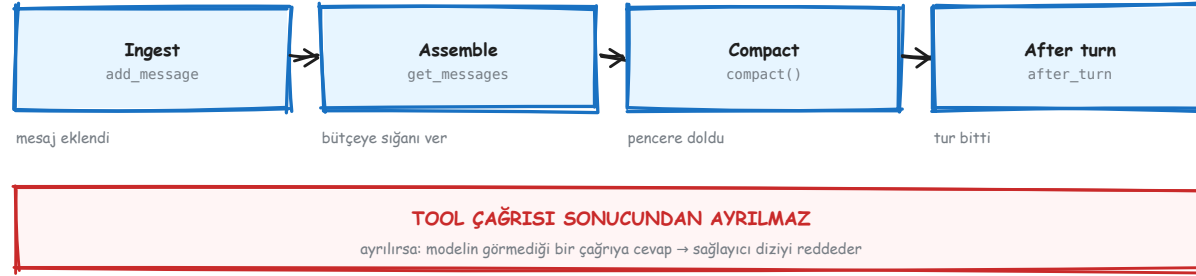
Döngü kırıcı ve sıkıştırma sonrası nöbetçi



aynı (tool, argüman, sonuç) üçlüsü tekrarlanırsa → koşu iptal

En pahalı hata sonsuz döngüdür, ve en sinsî biçimi şu zincirdir: bağlam doluyor, sıkıştırma çalışıyor, model sıkıştırma yüzünden ne yaptığını unutup aynı tool'u aynı argümanla tekrar çağırıyor, bağlam yine doluyor. Nöbetçi, sıkıştırmadan hemen sonra kısa bir pencere açıyor. O pencerede aynı (tool, argüman, sonuç) üçlüsü tekrarlanırsa koşuyu iptal ediyor. İnce ayar iki yönlü ve ikisi de gerekli. `exec` için hash hesaplanırken **oynak** alanlar (süre, PID, çalışma dizini) dışarıda bırakılıyor, yoksa aynı komut her seferinde farklı görünür ve döngü hiç yakalanmaz. Giden mesajlarda ise tam tersi yapılıyor: oynak id'ler çıkarılıyor ki iki ayrı gönderim yanlışlıkla aynı sanılmasın.

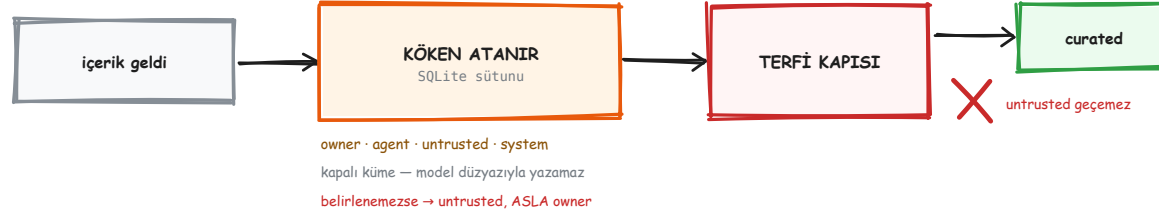
Bağlam motoru dört noktada değiştirilebilir



eklenti sözleşmesi – sıkıştırmanın kendisi bile takılabilir

Çoğu harness'ta "modele ne gönderilecek" kararı çekirdeğe gömülüdür ve dışarıdan değiştirilemez. OpenClaw'da bu kararın verildiği dört an da eklenti yüzeyi: mesaj bağlama eklenirken, model koşusundan hemen önce, pencere dolduğunda, ve tur bittiğinde. Sıkıştırma bile devralınabiliyor. Bir eklenti `ownsCompaction` bayrağını kaldırırsa sıkıştırmanın tamamı ona geçiyor. Bütün bu esnekliğin altında pazarlığa açık olmayan tek bir kural duruyor: **tool çağrısı sonucundan ayrılmaz**. Ayrılırsa model, cevabını hiç görmediği bir çağrı yapmış gibi görünür, ve çoğu sağlayıcı bu şekli geçersiz sayıp isteği reddeder.

Bellek bir sağlayıcı sözleşmesi



İki hijyen kuralı: cron/heartbeat/alt-ajan oturumları aday ÜRETMEZ · bellekten enjekte edilen içerik yeniden ÇIKARILMAZ.

"Yüz kez hatırlanan bir olgu tek bir olgu olarak kalır."

memory-host-sdk · 83 dosya · builtin / honcho / qmd

OpenClaw'da bellek tek bir uygulama değil. Bir **sözleşme** var, ve o sözleşmeyi uygulayan birden çok sağlayıcı: yerleşik olan, Honcho, QMD. Bunun için 83 dosyalık ayrı bir SDK paketi ayrılmış olması, işi ne kadar ciddiye aldıklarını gösteriyor. Sözleşmenin değişmeyen kısmı şu: her bellek kaydının bir **köken** sınıfı var, bu sınıf kapalı bir kümeden seçiliyor ve SQLite'ta ayrı bir sütunda duruyor. Yani **model onu düzyazıyla yazamıyor**. Sağlayıcı değişse de bu kural değişmiyor. Sonuç önemli: "belleği değiştirebilirsin" ile "güvenlik sınırını gevşetebilirsin" aynı şey değil. Genişletme noktası, sınırın *üstünde* duruyor.

Sessiz altyapı

ai · llm-core · model-catalog-core	sağlayıcı adaptörleri, akış runtime'ı	118
gateway-protocol · gateway-client · sdk	tipli şema + doğrulayıcı + istemci	108
memory-host-sdk	bellek sağlayıcı sözleşmesi	83
plugin-sdk · plugin-package-contract	eklenti yüzeyi	25
terminal-core · markdown-core · media-*	çıktı biçimlendirme	21
tool-call-repair · retry · net-policy	sessiz altyapı	6

22 paket · dosya sayıları en büyük modülü gösteriyor. Çekirdeğin her ilginç parçası ayrı paket — sıkıştırma bile.

`normalization-core · markdown-core · terminal-core · retry · net-policy`

Bir harness'ın görünmeyen yarısı bu beş pakette. Hiçbiri bir özellik listesinde yer almaz, ama biri eksik olduğunda hemen fark edilir. **normalization-core** farklı kanallardan gelen içeriği tek biçime indiriyor. **markdown-core** modelin yazdığını her kanalın kaldırabileceği biçime çeviriyor, çünkü WhatsApp'ın markdown'ı Slack'inki değil. **terminal-core** TUI çıktısını üretiyor. **retry** yeniden deneme politikasını tek dosyada topluyor. **net-policy** IP ayrıştırması, SSRF engellemesi ve URL redaksiyonu yapıyor. Sonuncusu iyi bir örnek: `net-policy` olmadan `web_fetch`, modelin eline verilmiş bir **iç ağ tarayıcısına** dönüşüyor.

Bizim harness'ımıza ne taşınır



Neden var: bazı modeller function calling'i düzgün üretmiyor, tool çağırısını metin olarak yazıyor.

Onarmadan atarsan tur boşa gider ve sebebi görünmez. Bu 6 dosyalık paket o turu kurtarıyor.

sıra: onarım → sonuç kısaltma → müdahale → trajectory

Bu destedeki mekanizmalardan dördü bugün `pipeline/` 'a girebilir, ve dördü de küçük iş. **Tool call repair** — küçük ya da yerel bir model kullanacaksak boşa giden turların bir kısmını kurtarır. **Sonuç ara katmanı** — bir `exec` çıktısının bağlamı yemesini engeller, ve zaten bir workbench sarmalayıcımız olduğu için tam oraya oturur. **Steer ve btw** — uzun koşullarda kullanıcıyı beklemekten kurtarır. **Trajectory** — panel zaten aşama yayınlıyor, onları tek bir pakete çevirmek küçük bir ek. Lobster ve self-learning ise daha büyük kararlar. İkisi de kontrol düzleminin olgunlaşmasını bekliyor.